

ACTIONSPlice: IN-FLIGHT ACTION EDITING FOR INTERACTIVE WORLD MODELS

Pardis Taghavi Tingyu Guo Jonas Lossner Gaurav Pandey Reza Langari
Texas A&M University

[Code](#) [Checkpoints](#) [Project Page](#)

Abstract. Chunk-autoregressive video world models typically condition each generated chunk on one action. An action received during sampling must therefore wait for the next chunk, condition future solver evaluations on a state produced under the previous action, or trigger rollback that repeats completed evaluations. We introduce *ActionSplice*, an inference framework that formulates this problem as Counterfactual State Transport (CST). A lightweight corrector transports the interrupted backbone-native representation toward the matched state induced by the revised action at the same solver step. The world model and sampler remain frozen, and sampling resumes without replaying completed evaluations. The retargeting variant CST_R updates the entire active chunk, while the temporal-splicing variant CST_T preserves a temporal prefix and updates only the suffix. Across minWM–Wan Action2V and HY-WM1.5, CST_R reduces rollback-relative LPIPS by 61.5% and 75.9% relative to direct condition swapping. CST_T reduces suffix LPIPS by 56.1% and 77.5%, respectively, while providing $2.73\times$ and $1.69\times$ pixel-ready speedups over waiting. Under the HY-WorldPlay protocol, CST_R obtains 25.66 PSNR, 0.6902 SSIM, and 0.1337 LPIPS against the original rollout.

1 INTRODUCTION

Interactive video world models must be both fast and responsive. Recent diffusion world models support action-conditioned visual simulation and real-time streaming generation (Valevski et al., 2025; Wang et al., 2026; Zhao et al., 2026a). Yet high frame throughput does not guarantee low control latency. A chunk-autoregressive model generates several latent frames under one action and denoises them through K evaluation steps before display. If a new action arrives after solver step r , the active state has already been shaped by the previous action. Waiting for the next chunk delays the visible response and limits control changes to the chunk rate. We study *interruptibility*, the ability to incorporate an action received during sampling into the active chunk while retaining completed computation and committed history.

The main obstacle is state misalignment. Waiting preserves the current trajectory but postpones the action. Direct condition swapping supplies the updated action to future evaluations, but leaves the active state on the trajectory induced by the previous action. Full rollback reconstructs the desired trajectory, but repeats the first r solver evaluations. This suggests a more useful target. At the interruption step, we seek the matched counterfactual state s_r^* that rollback would have reached under the revised action, with the same initial state, committed history, solver schedule, and stochastic inputs when applicable. A valid in-flight edit must approximate this state at the same solver step, preserve the non-editable portion of the transport representation, make the revised action visible within the active chunk, and avoid replaying completed evaluations.

We introduce *ActionSplice*, an inference framework built around Counterfactual State Transport (CST). CST learns a same-step correction from matched rollback pairs. Given an interrupted old-action state, a lightweight corrector predicts a masked residual in the backbone-native transport representation, reconstructs a valid solver state, and resumes the remaining $K - r$ evaluations with the world model and sampler frozen. The retargeting variant CST_R applies the revised action to the entire active chunk. The temporal-splicing variant CST_T preserves an old-action prefix and applies the revised action only to the remaining suffix, allowing control to change at finer temporal

granularity than one action per chunk. Full rollback is used only to construct training targets and evaluation references; it is not executed during ActionSplice inference.

CST does not assume a shared latent space or transfer corrector weights across models. Instead, it defines the same counterfactual intervention in each backbone’s resumable state space: clean-prediction transport for minWM–Wan Action2V (Zhao et al., 2026a) and Euler solver-state transport for HY-WM1.5 (HunyuanWorld, 2025). We train a separate corrector for each backbone and variant because the variants impose different intervention structures: CST_R transports the entire active state for asynchronous action revisions from a user or planner, whereas CST_T handles a scheduled action transition at a specified boundary within the active chunk.

Our contributions are threefold. First, we formulate in-flight action editing as matched counterfactual transport between resumable sampler states. Second, we introduce whole-chunk retargeting and within-chunk temporal splicing for frozen world models. Third, we establish a matched evaluation protocol across two independently implemented backbones that covers action receipt steps, directed action transitions, temporal boundaries, and repeated interruptions. Relative to direct condition swapping, CST_R reduces rollback-relative LPIPS by 61.5% on minWM–Wan Action2V and 75.9% on HY-WM1.5. CST_T reduces suffix LPIPS by 56.1% and 77.5%, respectively, while providing $2.73\times$ and $1.69\times$ pixel-ready speedups over waiting. On the HY-WorldPlay benchmark, CST_R obtains 25.66 PSNR, 0.6902 SSIM, and 0.1337 LPIPS against the original rollout, improving on the best reported comparison value for each metric.

2 RELATED WORK

Interactive video world models. Diffusion-based world modeling demonstrated action-conditioned visual simulation in interactive game environments (Alonso et al., 2024; Valevski et al., 2025). Recent systems target open-ended, camera-controllable streaming generation. WorldPlay uses dual action representations and reconstituted context memory for long-term geometric consistency (Sun et al., 2025), while Matrix-Game 3.0 combines long-horizon memory with multi-segment autoregressive distillation (Wang et al., 2026). HY-World 1.5 targets real-time interactive generation with geometric consistency (HunyuanWorld, 2025), while minWM and BiWM adapt bidirectional video diffusion backbones into camera-controllable autoregressive world models (Zhao et al., 2026a; Rui et al., 2026). Causal Forcing, Causal Forcing++, Self Forcing, and Causal-rCM improve the training and distillation of one- to four-step causal rollouts (Zhu et al., 2026; Zhao et al., 2026b; Huang et al., 2026; Zheng et al., 2026). These methods improve rollout quality, speed, and long-horizon consistency, but assume that the action remains fixed while a generation unit is sampled. ActionSplice addresses an action change during sampling by revising the active solver state rather than waiting for the next generation unit.

Interactive condition changes and memory. Existing methods handle condition changes through steering objectives, memory updates, or cache reconstruction. Delta Forcing constrains teacher-guided steering with a trust region to improve event responsiveness without destabilizing autoregressive rollout (Wu et al., 2026b). Echo-Forcing separates stable, recent, and recalled memory for prompt switching and scene recall (Wu et al., 2026a). LongLive refreshes cached states after prompt switches, while Anchor Forcing reconstructs the cache from anchor memories (Yang et al., 2025; 2026). Visko Orbis supports live prompt updates during long-form streaming generation (Gao et al., 2026). These approaches update future conditioning or attention memory, but do not estimate the denoising state that a revised action would induce at the current solver step. ActionSplice learns this counterfactual state from matched rollback pairs and resumes the remaining solver evaluations from the interruption point.

Inference-time reuse and acceleration. Methods for accelerating video diffusion reuse computation across denoising steps, attention operations, autoregressive chunks, and generation requests. TeaCache and FasterCache adaptively reuse model features across denoising steps. Pyramid Attention Broadcast reuses attention outputs over stable timestep intervals (Liu et al., 2025; Lyu et al., 2025; Zhao et al., 2025). Sparse VideoGen and SparsePR reduce attention cost through structured spatiotemporal sparsity (Xi et al., 2025; Taghavi et al., 2026). Recent methods extend reuse to autoregressive and interactive settings. Light Interaction combines adaptive context selection, denoising reuse, and sparse attention over blocks (Lu et al., 2026). X-Cache and C³ache reuse computation

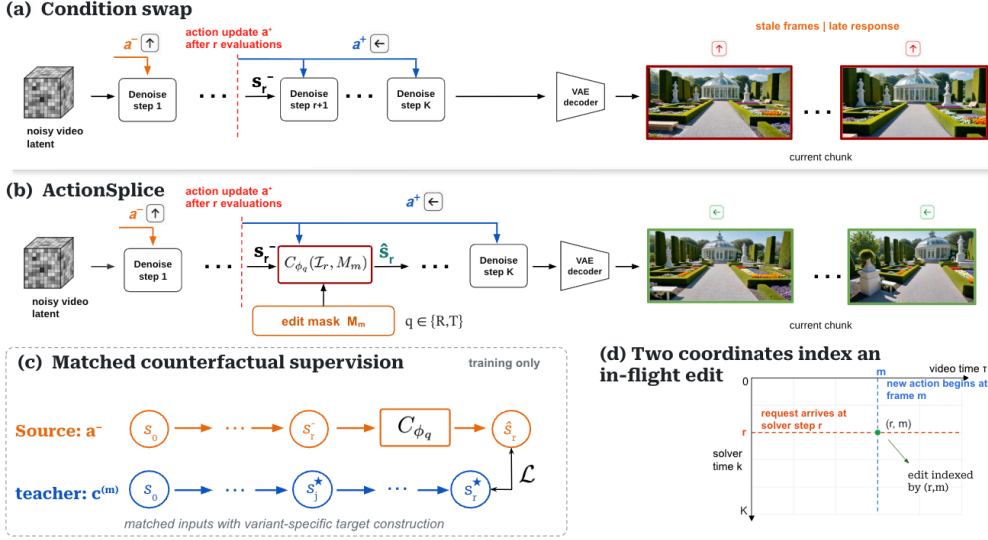


Figure 1: ActionSplice overview. (a) Direct condition swapping applies the revised action a^+ only to the remaining evaluations, leaving the interrupted state s_r^- on the trajectory induced by a^- . (b) At receipt step r , the corrector C_{ϕ_q} transports s_r^- to $\hat{s}_r$, after which the frozen sampler executes the remaining $K - r$ steps. CST_R uses $M_0 = 1$ to retarget the entire chunk. CST_T uses the suffix mask M_m to leave the prefix coordinates unchanged during transport. (c) Matched rollback provides the target s_r^* at the same solver step. CST_R uses ordinary new-action rollback, while CST_T clamps the old-action prefix after each replayed evaluation. Rollback is used only for training and evaluation. (d) The receipt step r indexes solver time, while the boundary m indexes video time.

across chunks, while Chorus reuses intermediate states across related requests (Zeng et al., 2026; Zhao et al., 2026c; Liu et al., 2026). These methods reuse computation that remains valid when conditioning does not change. An action update received during sampling makes the current solver state stale for the revised action. ActionSplice transports that state toward the trajectory induced by the new action and resumes the remaining solver evaluations without replaying completed ones.

Revisable denoising and editing of intermediate states. Diffusion Forcing uses independent noise levels for individual tokens to support flexible causal denoising, while Diffusion ReRoll selectively re-noises stable regions to revise predictions across a temporal horizon (Chen et al., 2024; Kim et al., 2026). SDEdit edits images by re-noising a source, EDICT recovers an editable diffusion trajectory through inversion, and Layered Diffusion Brushes caches intermediate latents for localized edits (Meng et al., 2021; Wallace et al., 2023; Gholami & Xiao, 2025). For video, FateZero preserves intermediate attention maps during inversion to guide temporally consistent edits (Qi et al., 2023). These methods revise generated content through flexible noise schedules, re-noising, inversion, or spatial masking. They do not estimate the active solver state that a revised action would have produced at the current step in a frozen world model. ActionSplice learns this transport from matched rollback pairs. Its temporally masked variant restricts transport to the editable suffix and places the action boundary inside the active chunk. Both variants resume from the interruption point without replaying completed solver evaluations.

3 METHOD

ActionSplice edits the state of an active chunk when the control changes before sampling is complete. Its core operation, Counterfactual State Transport (CST), uses a learned corrector to estimate the backbone-native transport state that the revised control would have produced at the current solver step. The world model and sampler remain frozen, and generation resumes from the corrected state without replaying completed solver evaluations. A valid in-flight edit must satisfy four requirements. It must remain at the current solver step, preserve committed history, make the updated action visible within the active chunk, and avoid replaying completed solver evaluations. We consider two settings

that differ in the temporal support of the correction. CST_R retargets the entire active chunk when an action arrives during solver execution. CST_T leaves the prefix coordinates of the transport representation unchanged and applies the revised action only to the suffix beginning at temporal boundary m . Figure 1 illustrates an overview of ActionSplice method.

3.1 PROBLEM FORMULATION

Let a frozen world model generate an active chunk of T latent frames with a K -step sampler. An action update arrives after $r \in \{1, \dots, K-1\}$ completed solver evaluations. We denote the backbone-native internal representation corrected by ActionSplice at solver step r by s_r^- . Depending on the sampler parameterization, s_r^- is either a cached clean prediction x_r^- or a solver state z_r^- . The state s_r^- lies on the trajectory induced by the previous action a^- , while the update requests a revised action a^+ .

Let m be the first latent position in the active chunk controlled by a^+ . The requested control schedule and editable mask are

$$c_\tau^{(m)} = \begin{cases} a^-, & \tau < m, \\ a^+, & \tau \geq m, \end{cases} \quad M_m[\tau] = \mathbf{1}[\tau \geq m], \quad \tau \in \{0, \dots, T-1\}. \quad (1)$$

The receipt step r identifies the interruption along the solver trajectory, while m identifies the action boundary along the temporal axis of the chunk. Setting $m = 0$ applies a^+ to the entire active chunk. For $0 < m < T$, latent positions before m remain under a^- and positions from m onward follow a^+ .

We define s_r^* as the matched counterfactual representation at solver step r under the requested control schedule $c^{(m)}$. For CST_R , full rollback constructs this target by replaying the first r solver evaluations under a^+ . For CST_T , we use the prefix-clamped rollback construction described in Section 3.3. The matched branches share the prompt, initial state, committed history, solver schedule, receipt step, and stochastic inputs when applicable. Direct condition swapping instead leaves s_r^- on the trajectory induced by a^- . The corrector must approximate s_r^* over the editable positions while preserving the remaining coordinates and resumability at step r .

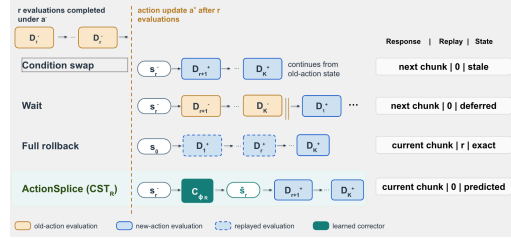


Figure 2: Action-update trade-offs after r of K solver evaluations. Condition swapping resumes from state s_r^- , waiting defers update, and full rollback replays r evaluations to recover s_r^* . CST_R predicts $\hat{s}_r$ and resumes without replay.

3.2 ACTIONSPICE: SAME-STEP STATE EDITING

ActionSplice predicts a residual that transports the interrupted representation s_r^- toward its matched counterfactual target s_r^* . Let $\mathcal{I}_r$ denote the backbone-specific context available at receipt step r . It includes s_r^- , the initial state, recent committed history, the previous and revised action conditioning, and any sampler information retained at the interruption. For recurrent editing, $\mathcal{I}_r$ may also encode the intervention history. For edit type $q \in \{R, T\}$, the corrector C_{ϕ_q} predicts a transport residual from $\mathcal{I}_r$ and M_m .

$$\hat{s}_r = s_r^- + M_m \odot C_{\phi_q}(\mathcal{I}_r, M_m), \quad (2)$$

The temporal mask is broadcast over the channel and spatial dimensions. It restricts the predicted residual to the requested action interval and leaves the remaining coordinates unchanged.

Whole-chunk editing. For CST_R , the updated action applies to the entire active chunk. We set $m = 0$ and $M_0 = \mathbf{1}$, allowing the corrector to update every active latent position. This variant supports asynchronous action revisions that should affect the full active chunk with the new control.

Prefix-preserving editing. For CST_T , the action changes at a temporal boundary $0 < m < T$. The hard output mask preserves positions $0, \dots, m-1$ and restricts the correction to positions $m, \dots, T-1$. The corrected representation therefore retains the old-action prefix and applies the new action to the suffix of the same active chunk. This variant supports scheduled action transitions at a specified boundary within the active chunk.

State reconstruction and resumption. The corrected representation $\hat{s}_r$ is inserted into a valid solver state at the same solver step r . For minWM, $\hat{s}_r = \hat{x}_r$ is the corrected clean prediction. The native scheduler reconstructs $\hat{z}_r$ from $\hat{x}_r$ and the stored transition noise. For HY-WM1.5, $\hat{s}_r = \hat{z}_r$ is the corrected Euler solver state, so no reconstruction is required. ActionSplice invalidates temporary action-dependent cache entries while preserving committed-history entries. Sampling then resumes from $\hat{z}_r$ under $c^{(m)}$ for the remaining $K - r$ backbone evaluations.

After the update, ActionSplice performs one corrector evaluation and $K - r$ backbone evaluations. Full rollback instead performs K backbone evaluations, comprising r replayed evaluations and the remaining $K - r$ evaluations. ActionSplice therefore avoids replaying the r backbone evaluations completed before the update, separately from backbone evaluations.

3.3 MATCHED COUNTERFACTUAL SUPERVISION

Each training example pairs an interrupted source representation s_r^- with the matched counterfactual target s_r^* defined in Section 3.1. The source and target branches share the prompt, initial state, committed history, solver schedule, receipt step, and stochastic inputs when applicable. They differ only in the action schedule applied to the active chunk. This pairing isolates the change in the backbone-native representation induced by the action update.

For CST_R , the source branch reaches solver step r under a^- . The matched rollback branch replays the first r solver evaluations from the same initial state under a^+ . Its backbone-native representation at step r defines s_r^* .

For CST_T , we construct the target using matched prefix-clamped rollback. The target branch starts from the same initial state as the source branch and is evaluated under the requested-action conditioning. After every replayed solver evaluation, we replace its latent positions $0, \dots, m-1$ with the corresponding source-branch positions and retain the updated suffix $m, \dots, T-1$. The source and target representations therefore agree exactly before the temporal boundary:

$$s_r^*[\tau] = s_r^-[\tau], \quad \tau < m. \quad (3)$$

Consequently, the target correction is identically zero over the clamped prefix. The transport residual and its masked training objective are defined in Section 3.4.

Recurrent training exposes each corrector to histories produced by its earlier predictions. After each student correction, sampling completes the active chunk and the rollout continues to the next interruption. At each interruption, the matched source and teacher branches share the same student-produced committed history. The teacher uses ordinary new-action rollback for CST_R and prefix-clamped rollback for CST_T . This construction reduces the mismatch between training and recurrent inference. Rollback is used only to construct supervision and evaluation references and is not executed during ActionSplice inference.

3.4 TRANSPORT CORRECTOR AND TRAINING OBJECTIVES

We train a separate corrector for each backbone and edit type. Each corrector is a six-block residual 3D encoder-decoder. FiLM conditioning (Perez et al., 2018) modulates its hidden features using embeddings of the camera controls and solver step. For CST_T , the corrector also receives the temporal mask M_m . Applying this mask to the predicted correction enforces exact preservation of the prefix at the receipt step.

We measure transport error over the editable elements using masked normalized mean squared error:

$$\text{NMSE}_M(u, v) = \frac{\|M \odot (u - v)\|_2^2 / N_M}{\max(\|M \odot v\|_2^2 / N_M, \epsilon)}, \quad (4)$$

where N_M is the number of selected scalar elements after broadcasting the temporal mask over channels and spatial locations. We write NMSE for the unmasked case $M = 1$.

Let $\widehat{\Delta s_r} = \widehat{s_r} - s_r^-$ denote the predicted transport and $\Delta s_r^* = s_r^* - s_r^-$ its matched target. State and residual matching have the same error numerator because $\widehat{\Delta s_r} - \Delta s_r^* = \widehat{s_r} - s_r^*$. We use the residual form because its normalization measures error relative to the required action-induced transport. The transport loss is $\mathcal{L}_{\text{tr}} = \text{NMSE}_M(\widehat{\Delta s_r}, \Delta s_r^*)$.

For CST_T , let $\widehat{d}_m = \widehat{s_r}[m] - \widehat{s_r}[m-1]$ and $d_m^* = s_r^*[m] - s_r^*[m-1]$ denote the predicted and target changes across the temporal boundary. We define $\mathcal{L}_{\text{mid}} = \text{NMSE}(\widehat{d}_m, d_m^*)$. This term is omitted for CST_R . The core transport objective is

$$\mathcal{L} = \mathcal{L}_{\text{tr}} + \lambda_{\text{mid}} \mathcal{L}_{\text{mid}}, \quad (M, \lambda_{\text{mid}}) = \begin{cases} (1, 0), & \text{CST}_R, \\ (M_m, \lambda_m), & \text{CST}_T. \end{cases} \quad (5)$$

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP AND EVALUATION PROTOCOL

We evaluate ActionSplice on minWM–Wan Action2V (Zhao et al., 2026a) and HY-WM1.5 (Hun-yuanWorld, 2025). Both backbones use a four-step sampler, and we train a separate corrector for each backbone and CST variant. We use the same prompt-disjoint split of 150 scene prompts for both backbones, with 120 for training and 30 reserved for evaluation. Each prompt yields three interruption captures per corrector, giving 360 training and 90 held-out captures. Training captures for both variants come from recurrent rollouts, so later interruptions inherit committed histories produced after earlier corrections. Each CST_R training trajectory contains three action updates and four action segments, while CST_T captures are balanced over temporal boundaries $m \in \{1, 2, 3\}$. Both partitions are balanced across the six directed transitions among forward, backward, and yaw-left actions. The main evaluation uses receipt step $r = 2$ and contains 30 matched prompt groups for CST_R and 90 prompt-boundary groups for CST_T . The appendix reports results for $r \in \{1, 2, 3\}$.

We compare against waiting, condition swapping, partial rollback, re-noising, and a matched rollback oracle. Waiting applies the revised action in the next chunk, while condition swapping changes the action for the remaining evaluations without correcting the interrupted state. Partial rollback restores the preceding solver checkpoint and replays one completed evaluation. Re-noising forms a clean estimate from the interrupted state, re-noises it to the noise level at the start of the final two solver evaluations, and reruns those evaluations under the revised action. The matched oracle uses ordinary full rollback for CST_R and prefix-clamped rollback for CST_T . Within each comparison group, all methods share the prompt, initial state, committed history, action update, receipt step, and solver schedule. The initial stochastic inputs are matched, while re-noising uses an additional fixed Gaussian draw.

Action following is the percentage of examples in which the revised action appears within the intended editable region. Stale frames count decoded frames in that region before the first visible response. Both measurements use human annotations blinded to method identity and are reported in Table 3. Figure 6 shows the annotation interface. We compute LPIPS and PSNR against the matched counterfactual reference over the full active chunk for CST_R and over the suffix $\tau \geq m$ for CST_T . Prefix LPIPS measures preservation before m . Boundary error is measured at the history-to-active transition for CST_R and between positions $m-1$ and m for CST_T . Latent-ready latency ends when the responsive latent is available. Pixel-ready latency ends when the first responsive frame is decoded. Unless otherwise stated, we average framewise metrics within each example and report the median across matched examples. HY-WorldPlay quality and camera-trajectory metrics follow their benchmark protocols and are reported as arithmetic means across examples. Human action following is reported as a proportion, and stale frames as the mean response delay. Timing is measured on an NVIDIA A100 for minWM and an NVIDIA H100 for HY-WM1.5, with runtime comparisons restricted to the same backbone. Optimization and checkpoint-selection details are provided in the appendix. Code and trained corrector weights are open source.

Table 1: Whole-chunk action-update results after $r = 2$ of $K = 4$ solver evaluations over 30 held-out prompts. Fidelity metrics compare the active chunk with matched full rollback. Speedup is computed from pixel-ready latency relative to waiting. Bold and underlined values indicate best and second-best non-oracle results within each backbone.

Model	Method	Latent-ready latency (ms) ↓	Pixel-ready latency (ms) ↓	Speedup ↑	LPIPS ↓	PSNR ↑	Boundary error ↓
minWM	Wait	5295.8	6815.0	1.00×	0.3159	12.95	0.0215
	Condition Swap	979.2	2498.1	2.73×	0.3155	<u>12.97</u>	0.0215
	Partial Rollback- $d = 1$	1467.4	2979.3	2.29×	<u>0.3111</u>	12.95	<u>0.0212</u>
	Re-noising (rerun 2)	<u>983.4</u>	2506.4	<u>2.72×</u>	0.3160	12.92	0.0214
	Full Rollback	1961.0	3481.4	1.96×	—	—	—
	CST_R	1016.2	<u>2498.3</u>	2.73×	0.1214	16.30	0.0175
HY-WM1.5	Wait	7414.0	11711.2	1.00×	0.3682	14.94	0.0379
	Condition Swap	2820.7	7052.1	1.66×	0.3446	15.20	<u>0.0368</u>
	Partial Rollback- $d = 1$	3619.0	7942.8	1.47×	<u>0.1674</u>	<u>18.89</u>	0.0399
	Re-noising	<u>2818.7</u>	<u>7101.2</u>	<u>1.65×</u>	0.3480	15.08	0.0379
	Full Rollback	4415.0	8706.8	1.34×	—	—	—
	CST_R	2748.8	7442.2	1.57×	0.0831	20.51	0.0196

4.2 WHOLE-CHUNK ACTION UPDATES

We evaluate whole-chunk retargeting with an action update received after $r = 2$ of the $K = 4$ solver evaluations. The revised action should control the entire active chunk. Table 1 compares CST_R with waiting, condition swapping, partial rollback, re-noising, and matched full rollback. All methods use the matched protocol described above. Full rollback provides the counterfactual reference but replays the completed evaluations. Receipt-step sweeps and results for all six directed action transitions are reported in Appendices B.1 and B.3, respectively.

CST_R provides the best rollback-relative fidelity among the non-oracle methods on both backbones. On minWM, it matches the pixel-ready latency of condition swapping while reducing LPIPS by 61.5% and improving PSNR by 3.33 dB. On HY-WM1.5, it reduces LPIPS by 75.9% and improves PSNR by 5.31 dB relative to condition swapping, while remaining 1.57× faster than waiting. It also obtains the lowest non-oracle boundary error on both backbones. These results show that same-step transport approaches rollback fidelity without replaying the completed solver evaluations.

4.3 WITHIN-CHUNK ACTION TRANSITIONS

We evaluate action updates intended to take effect at temporal boundary m within the active chunk. Positions before m should retain the previous action, while positions from m onward should follow the revised action. Only CST_T explicitly restricts its correction to this suffix. Table 2 reports aggregate results, while Appendix B.2 reports results for each boundary.

Table 2: Within-chunk action transitions at receipt step $r = 2$ with $K = 4$, aggregated over 90 matched groups from 30 prompts and boundaries $m \in \{1, 2, 3\}$. Fidelity is measured against prefix-clamped rollback, and speedup uses pixel-ready latency relative to waiting. Bold and underlined values indicate the best and second-best non-oracle results within each backbone.

Model	Method	Latent-ready latency (ms) ↓	Pixel-ready latency (ms) ↓	Speedup ↑	Prefix LPIPS ↓	Suffix LPIPS ↓	Suffix PSNR ↑	Boundary error ↓
minWM	Wait	5292.5	6811.6	1.00×	0.0863	0.2874	13.15	0.0552
	Condition Swap	979.2	2494.4	2.73×	0.0867	0.2868	13.16	0.0548
	Partial Rollback- $d = 1$	1468.4	2986.6	2.28×	<u>0.0842</u>	<u>0.2773</u>	<u>13.27</u>	<u>0.0542</u>
	Re-noising (rerun 2)	<u>982.4</u>	2502.7	<u>2.72×</u>	0.0897	0.2889	13.14	0.0549
	Full Rollback (oracle)	1962.6	3482.4	1.96×	—	—	—	—
	CST_T	1013.8	<u>2496.4</u>	2.73×	0.0740	0.1258	16.46	0.0511
HY-WM1.5	Wait	7406.3	11701.7	1.00×	0.0715	0.3612	15.06	0.0497
	Condition Swap	2818.2	7132.4	<u>1.64×</u>	0.0763	0.2731	15.53	0.0451
	Partial Rollback- $d = 1$	3618.1	7901.2	1.48×	<u>0.0498</u>	<u>0.0809</u>	<u>21.52</u>	<u>0.0269</u>
	Re-noising (rerun 2)	<u>2811.8</u>	<u>7115.5</u>	<u>1.64×</u>	0.0933	0.2790	15.51	0.0464
	Full Rollback (oracle)	6012.7	10305.8	1.14×	—	—	—	—
	CST_T	2746.3	6941.2	1.69×	0.0412	0.0615	25.05	0.0215

Table 3: Human evaluation of action responsiveness on HY-WM1.5 over 30 matched CST_R groups and 90 matched CST_T groups. Action following measures responses within the editable region; mean stale frames measures the delay to the first visible response.

Model	Method	CST_R		CST_T	
		Action following (%) $\uparrow$	Mean stale frames $\downarrow$	Action following (%) $\uparrow$	Mean stale frames $\downarrow$
HY-WM1.5	Wait	21.1	15.37	26.7	7.53
	Condition Swap	84.2	8.89	86.7	3.73
	Partial Rollback- $d = 1$	<u>89.5</u>	<u>3.00</u>	<u>93.3</u>	<u>0.93</u>
	Re-noising	78.9	9.89	86.7	4.40
	Full Rollback (oracle)	100.0	0.00	100.0	0.00
	CST	94.7	1.05	100.0	0.40

CST_T obtains the best non-oracle prefix preservation, suffix fidelity, and boundary continuity on both backbones. On minWM, it matches the pixel-ready latency of condition swapping while reducing suffix LPIPS by 56.1% and improving suffix PSNR by 3.30 dB. On HY-WM1.5, it reduces suffix LPIPS by 77.5% and improves suffix PSNR by 9.52 dB relative to condition swapping, while providing a $1.69\times$ speedup over waiting. Its lower prefix LPIPS and boundary error indicate better prefix preservation and a smoother transition at m .

4.4 COMPARISON WITH PRIOR WORK

Table 4 compares CST_R with published HY-WorldPlay results (Lu et al., 2026). This complementary benchmark measures global video quality rather than the in-flight action-update fidelity and latency in Tables 1 and 2. Our result uses a reproduced HY-WM1.5 backbone under the same protocol.

Table 4: Global video quality under the HY-WorldPlay protocol. CST_R uses reproduced HY-WM1.5. Bold and underlined values indicate the best and second-best reported results.

Model	Method	vs. Original			Self-Comparison		
		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
HY-WorldPlay	Original (HunyuanWorld, 2025)	—	—	—	18.60	0.5678	0.2051
	SVG (Xi et al., 2025)	19.48	0.6028	0.2209	17.75	0.5299	0.2187
	BSA (Meituan LongCat Team et al., 2025)	15.94	0.4639	0.3755	15.44	0.4205	0.3720
	TeaCache (Liu et al., 2025)	20.90	<u>0.6588</u>	0.1892	<u>18.86</u>	0.5743	0.2054
	Light Interaction (Lu et al., 2026)	<u>24.81</u>	0.6500	<u>0.1788</u>	18.85	<u>0.5854</u>	<u>0.1963</u>
	CST_R	25.66	0.6902	0.1337	20.38	0.6267	0.1587

CST_R obtains the best reported PSNR, SSIM, and LPIPS in both comparisons, indicating that in-flight state transport preserves global video quality under the established benchmark.

4.5 CAMERA-ACTION FOLLOWING

We evaluate camera-action following with the rotational and translational trajectory errors R_{dist} and T_{dist} from HunyuanWorld (2025). ViPE estimates camera poses from each video. We express the estimated and target poses relative to their first frames and normalize translation scale. We score the active chunk for CST_R and only the editable suffix $\tau \geq m$ for CST_T . Table 5 reports published short-term references and our in-flight results. The request protocols differ, so the published rows provide context rather than a direct ranking.

4.6 REPEATED IN-FLIGHT ACTION UPDATES

We test whether transport error accumulates under repeated updates. Each HY-WM1.5 rollout contains six action segments separated by five updates, all received after solver step $r = 2$ of $K = 4$. The CST_R corrector is trained on trajectories with three interruptions and evaluated on five-interruption sequences without further training. Condition swapping and sequential Full Rollback use matched prompts, action schedules, initial states, and solver trajectories. Figure 3 reports rotation and translation error, rollback-relative LPIPS, and boundary error after each interruption.

Table 5: Camera-action following measured by rotation and translation trajectory error. Published short-term references follow HunyuanWorld (2025). Lower is better.

Method	$R_{\text{dist}} \downarrow$	$T_{\text{dist}} \downarrow$
CameraCtrl	0.037	0.341
VMem	0.048	0.219
Matrix-Game 2.0 (He et al., 2025)	0.287	0.843
WorldPlay (HunyuanWorld, 2025)	0.031	0.121
CST_R	0.037	0.049
CST_T	0.035	0.120

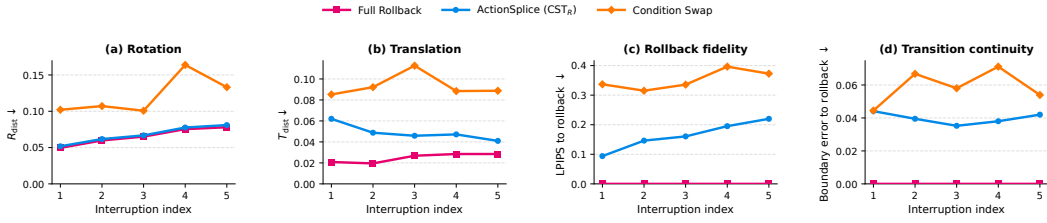


Figure 3: Repeated in-flight updates on HY-WM1.5. A CST_R corrector trained on three-interruption trajectories is applied at five interruptions without further training. Curves show mean error over the evaluation prompts. Sequential Full Rollback provides the reference.

Across all interruption indices, CST_R remains closer to sequential Full Rollback than condition swapping on every metric.

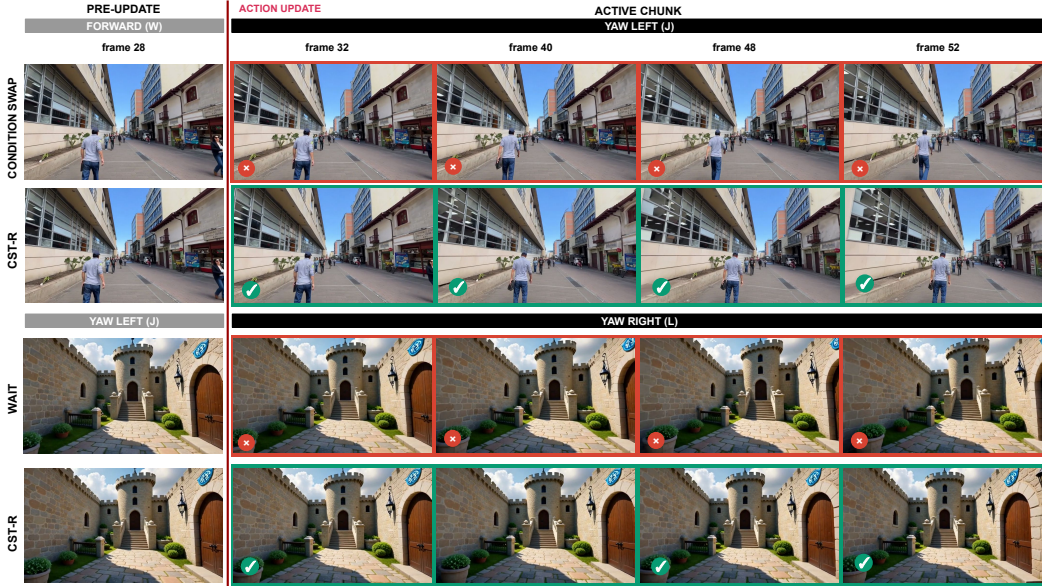


Figure 4: Qualitative whole-chunk action updates. The top pair shows HY-WM1.5 changing from forward to yaw left. Condition swapping retains stale forward motion, while CST_R responds within the active chunk. The bottom pair shows minWM changing from yaw left to yaw right. Waiting defers the request, while CST_R responds in the active chunk. Red and green borders mark stale and responsive frames.

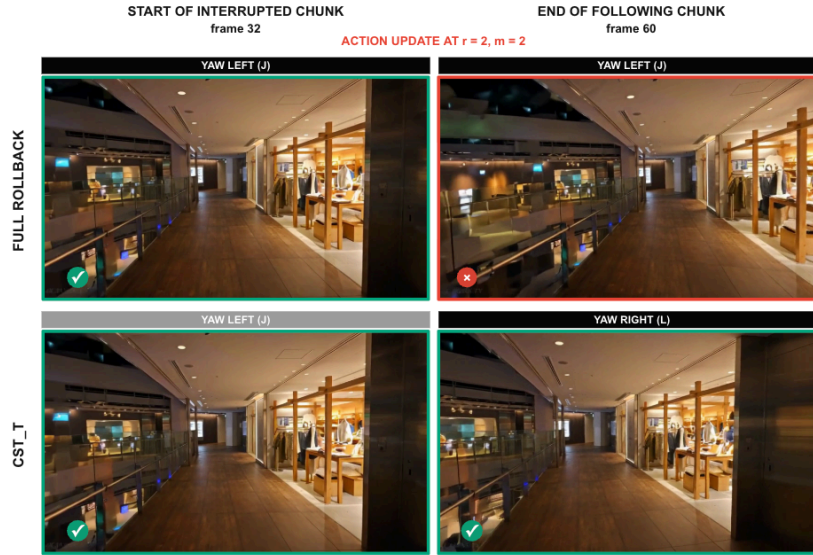


Figure 5: Qualitative within-chunk transition on HY-WM1.5. The update arrives at $r = 2$ and requests yaw right from $m = 2$. The Full Rollback row follows the original yaw-left action throughout the chunk, while CST_T preserves the yaw-left prefix and applies yaw right to the suffix. Both rollouts continue through the following chunk to make the trajectory difference visible. The yaw-right target is outside the corrector’s training action set and is shown only as a qualitative generalization example.

4.7 QUALITATIVE COMPARISONS

Figure 4 shows representative whole-chunk updates on both backbones. On HY-WM1.5, condition swapping continues the forward trajectory after a yaw left request, while CST_R turns within the active chunk. On minWM, waiting defers a yaw right request, while CST_R applies it within the active chunk. In both cases, same-step transport makes the revised action visible without restarting the chunk.

Figure 5 complements the within-chunk results in Table 2. An update received at $r = 2$ requests a yaw-left-to-yaw-right transition at $m = 2$. A one-action rollback follows yaw left throughout the chunk, whereas CST_T preserves the prefix and applies yaw right to the suffix. We continue both rollouts through the following chunk to make the trajectory difference visible.

4.8 ABLATIONS

We ablate matched counterfactual supervision and the hard output mask under the full-model evaluation protocol. Removing the mask tests whether CST_T can learn prefix preservation without an explicit output constraint. Unmatched supervision removes the matched source-teacher pairing. Table 6 reports action error, rollback-relative fidelity, boundary continuity, and prefix preservation.

Table 6: ActionSplice ablations. RB-LPIPS measures fidelity to matched Full Rollback, and prefix LPIPS applies only to CST_T . Lower is better for all metrics.

Configuration	CST_R			CST_T			
	Action error ↓	RB-LPIPS ↓	Boundary error ↓	Action error ↓	RB-LPIPS ↓	Boundary error ↓	Prefix LPIPS ↓
Full model	0.6192	0.0831	0.0196	0.3283	0.0615	0.0215	0.0412
Without hard mask	—	—	—	0.4092	0.0787	0.0240	0.0604
Unmatched supervision	4.6889	0.7797	0.0435	3.0415	0.7719	0.0349	0.4209

The full model performs best on every reported metric for both variants. Removing the hard mask worsens CST_T prefix preservation and boundary continuity. Unmatched supervision degrades every metric, supporting the use of paired states that differ only in the requested action schedule.

5 CONCLUSION

We introduced ActionSplice for same-step action updates during diffusion sampling. CST_R retargets the active chunk, while CST_T preserves a temporal prefix and updates only the suffix. Across minWM and HY-WM1.5, matched counterfactual transport improves rollback-relative fidelity over direct condition swapping without replaying completed evaluations. It also remains effective under repeated interruptions and preserves global video quality under the HY-WorldPlay protocol.

REFERENCES

- Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. *Advances in Neural Information Processing Systems*, 37:58757–58791, 2024.
- Boyuan Chen, Diego Martí Monsó, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. *Advances in Neural Information Processing Systems*, 37:24081–24125, 2024.
- Xiangbo Gao, Siyuan Yang, Ping He, Mingyang Wu, Yuheng Wu, Yushen Zuo, Jiongze Yu, Ryan Cui, Hongyuan Hua, Devin Ma, et al. Visko orbis 1.0: A live model for real-time interactive long video generation. *arXiv preprint arXiv:2607.26694*, 2026.
- Peyman Gholami and Robert Xiao. Streamlining image editing with layered diffusion brushes. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 17368–17378. IEEE, 2025.
- Xianglong He, Chunli Peng, Zexiang Liu, Boyang Wang, Yifan Zhang, Qi Cui, Fei Kang, Biao Jiang, Mengyin An, Yangyang Ren, et al. Matrix-game 2.0: An open-source real-time and streaming interactive world model. *arXiv preprint arXiv:2508.13009*, 2025.
- Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. *Advances in Neural Information Processing Systems*, 38:167283–167308, 2026.
- Team HunyuanWorld. Hy-world 1.5: A systematic framework for interactive world modeling with real-time latency and geometric consistency. *arXiv preprint*, 2025.
- Seonsoo Kim, Seongil Hong, and Jun-Gill Kang. Diffusion reroll: Revisable denoising for robotic sequential prediction. *arXiv preprint arXiv:2607.19919*, 2026.
- Feng Liu, Shiwei Zhang, Xiaofeng Wang, Yujie Wei, Haonan Qiu, Yuzhong Zhao, Yingya Zhang, Qixiang Ye, and Fang Wan. Timestep embedding tells: It’s time to cache for video diffusion model. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7353–7363. IEEE, 2025.
- Hao Liu, Ye Huang, Chenghuan Huang, Zhenyi Zheng, Jiangsu Du, Ziyang Ma, Jing Lyu, and Yutong Lu. Beyond few-step inference: Accelerating video diffusion transformer model serving with inter-request caching reuse. *arXiv preprint arXiv:2604.04451*, 2026.
- Jiacheng Lu, Haoyi Zhu, Sipei Yi, Enze Xie, Yu Li, and Cheng Zhuo. Light interaction: Training-free inference acceleration for interactive video world models. *arXiv preprint arXiv:2605.31158*, 2026.
- Zhengyao Lyu, Chenyang Si, Junhao Song, Zhenyu Yang, Yu Qiao, Ziwei Liu, and Kwan-Yee K Wong. Fastercache: Training-free video diffusion model acceleration with high quality. In *International Conference on Learning Representations*, volume 2025, pp. 33132–33156, 2025.

- Meituan LongCat Team, Xunliang Cai, Qilong Huang, Zhuoliang Kang, Hongyu Li, Shijun Liang, Liya Ma, Siyu Ren, Xiaoming Wei, Rixu Xie, and Tong Zhang. Longcat-video technical report. *arXiv preprint arXiv:2510.22200*, 2025. URL <https://arxiv.org/abs/2510.22200>.
- Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. Sdedit: Guided image synthesis and editing with stochastic differential equations. *arXiv preprint arXiv:2108.01073*, 2021.
- Ethan Perez, Florian Strub, Harm De Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- Chenyang Qi, Xiaodong Cun, Yong Zhang, Chenyang Lei, Xintao Wang, Ying Shan, and Qifeng Chen. Fatezero: Fusing attentions for zero-shot text-based video editing. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 15886–15896. IEEE, 2023.
- Shaohao Rui, Xiaofeng Mao, Zhanyu Zhang, Peijia Lin, Yansong Zhu, Yibo Zhang, Haibin Wan, Zhangrui Zhao, and Weijie Ma. Biwm: Advancing open-source interactive video world models with bidirectional autoregression. *arXiv preprint arXiv:2606.10135*, 2026.
- Wenqiang Sun, Haiyu Zhang, Haoyuan Wang, Junta Wu, Zehan Wang, Zhenwei Wang, Yunhong Wang, Jun Zhang, Tengfei Wang, and Chunchao Guo. Worldplay: Towards long-term geometric consistency for real-time interactive world modeling. *arXiv preprint arXiv:2512.14614*, 2025.
- Pardis Taghavi, Reza Langari, and Gaurav Pandey. Partition the support, reconstruct the residual: Training-free sparse attention for video generation and world models. *arXiv preprint arXiv:2608.18484*, 2026.
- Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. In *International Conference on Learning Representations*, volume 2025, pp. 73754–73776, 2025.
- Bram Wallace, Akash Gokul, and Nikhil Naik. Edict: Exact diffusion inversion via coupled transformations. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 22532–22541. IEEE, 2023.
- Zile Wang, Zexiang Liu, Jiaxing Li, Kaichen Huang, Baixin Xu, Fei Kang, Mengyin An, Peiyu Wang, Biao Jiang, Yichen Wei, et al. Matrix-game 3.0: Real-time and streaming interactive world model with long-horizon memory. *arXiv preprint arXiv:2604.08995*, 2026.
- Mingqiang Wu, Weilun Feng, Zhefeng Zhang, Haotong Qin, Yuqi Li, Guoxin Fan, Xiaokun Liu, Zhulin An, Libo Huang, Yongjun Xu, et al. Echo-forcing: A scene memory framework for interactive long video generation. *arXiv preprint arXiv:2605.16003*, 2026a.
- Yuheng Wu, Xiangbo Gao, Tianhao Chen, Xinghao Chen, Qing Yin, Zhengzhong Tu, and Dongman Lee. Delta forcing: Trust region steering for interactive autoregressive video generation. *arXiv preprint arXiv:2605.14382*, 2026b.
- Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, et al. Sparse videogen: Accelerating video diffusion transformers with spatial-temporal sparsity. *arXiv preprint arXiv:2502.01776*, 2025.
- Shuai Yang, Wei Huang, Ruihang Chu, Yicheng Xiao, Yuyang Zhao, Xianbang Wang, Muyang Li, Enze Xie, Yingcong Chen, Yao Lu, et al. Longlive: Real-time interactive long video generation. *arXiv preprint arXiv:2509.22622*, 2025.
- Yang Yang, Tianyi Zhang, Wei Huang, Jinwei Chen, Boxi Wu, Xiaofei He, Deng Cai, Bo Li, and Peng-Tao Jiang. Anchor forcing: Anchor memory and tri-region rope for interactive streaming video diffusion. *arXiv preprint arXiv:2603.13405*, 2026.
- Yixiao Zeng, Jianlei Zheng, Chaoda Zheng, Shijia Chen, Mingdian Liu, Tongping Liu, Tengwei Luo, Yu Zhang, Boyang Wang, Linkun Xu, et al. X-cache: Cross-chunk block caching for few-step autoregressive world models inference. *arXiv preprint arXiv:2604.20289*, 2026.

- Min Zhao, Hongzhou Zhu, Bokai Yan, Zihan Zhou, Yimin Chen, Wenqiang Sun, Kaiwen Zheng, Guande He, Xiao Yang, Chongxuan Li, et al. minwm: A full-stack open-source framework for real-time interactive video world models. *arXiv preprint arXiv:2605.30263*, 2026a.
- Min Zhao, Hongzhou Zhu, Kaiwen Zheng, Zihan Zhou, Bokai Yan, Xinyuan Li, Xiao Yang, Chongxuan Li, and Jun Zhu. Causal forcing++: Scalable few-step autoregressive diffusion distillation for real-time interactive video generation. *arXiv preprint arXiv:2605.15141*, 2026b.
- Weisen Zhao, Lam Nguyen, Zhicong Lu, and Yuzhang Shang. C³ache: Accelerating world action models with cross inference chunk cache. *arXiv preprint arXiv:2606.08962*, 2026c.
- Xuanlei Zhao, Xiaolong Jin, Kai Wang, and Yang You. Real-time video generation with pyramid attention broadcast. In *International Conference on Learning Representations*, volume 2025, pp. 3296–3319, 2025.
- Kaiwen Zheng, Guande He, Min Zhao, Jintao Zhang, Huayu Chen, Jianfei Chen, Chen-Hsuan Lin, Ming-Yu Liu, Jun Zhu, and Qianli Ma. Causal-rcm: A unified teacher-forcing and self-forcing open recipe for autoregressive diffusion distillation in streaming video generation and interactive world models. *arXiv preprint arXiv:2606.25473*, 2026.
- Hongzhou Zhu, Min Zhao, Guande He, Hang Su, Chongxuan Li, and Jun Zhu. Causal forcing: Autoregressive diffusion distillation done right for high-quality real-time interactive video generation. *arXiv preprint arXiv:2602.02214*, 2026.

SUPPLEMENTARY MATERIAL

A IMPLEMENTATION AND TRAINING DETAILS

A.1 BACKBONE-SPECIFIC TRANSPORT AND RESUMPTION

For minWM–Wan Action2V, the corrector edits the cached clean prediction $x_r = \text{pred}_x 0$. We reconstruct z_r with the stored transition-noise tensor and the native scheduler. For HY-WM1.5, the corrector edits z_r directly and resumes deterministic Euler sampling at the same solver step.

A.2 DATASET CONSTRUCTION

We use a fixed prompt-disjoint split of 150 prompt-defined scenes. The split contains 120 training prompts and 30 held-out prompts, with one fixed seed per prompt. The same master manifest and split are used for minWM and HY-WM1.5. Training captures are balanced across the six directed transitions among forward, backward, and yaw-left motion. Each capture stores the same solver trace at receipt steps $r \in \{1, 2, 3\}$.

For CST_R , each prompt produces one recurrent trajectory with three interruption events and four action segments. Each action persists for three or four chunks, which produces trajectories of 48 to 64 latent frames. The three events yield 450 captures in total, split into 360 training and 90 held-out captures. For CST_T , each prompt produces three recurrent captures with temporal boundaries $m \in \{1, 2, 3\}$. The split again contains 360 training and 90 held-out captures. The six action transitions and three temporal boundaries are balanced in both partitions.

A.3 OPTIMIZATION, LOSSES, AND CHECKPOINT SELECTION

We train each corrector for 4,000 steps with AdamW, batch size 1, weight decay 10^{-4} , and gradient clipping at 1.0. The learning rate follows a cosine schedule from 10^{-5} to 10^{-6} . The state-matching term has unit weight for both backbones. minWM adds a unit-weight residual term, while HY-WM1.5 directly matches the solver state and sets the residual-loss weight to zero. Both backbones use decoded LPIPS, temporal, and history-to-chunk boundary losses with weights 0.05, 0.1, and 0.1. CST_T adds an intra-chunk boundary loss with weight 0.1. These auxiliary terms complement the core transport objective in Eq. equation 5.

We initialize the minWM correctors from their corresponding CST_R and multi-boundary CST_T pilot checkpoints. Both HY-WM1.5 correctors are initialized from the HY-WM1.5 pilot corrector. The world model and sampler remain frozen. We maintain an exponential moving average of the corrector weights with decay 0.999 and evaluate it every 200 steps. The reported checkpoint is the EMA checkpoint with the lowest held-out transport NMSE.

B ADDITIONAL RESULTS

B.1 RECEIPT-STEP SENSITIVITY

We vary the CST_R receipt step $r \in \{1, 2, 3\}$ on HY-WM1.5 while holding the prompt, initial state, action transition, solver schedule, and stochastic inputs fixed. All rows use the same 30 held-out prompts and matched trajectories as the main evaluation. Table 7 measures remaining computation, latency, rollback fidelity, boundary continuity, and camera-trajectory error.

Table 7: Receipt-step sweep for CST_R on HY-WM1.5. The sampler uses $K = 4$, so Action-Splice executes $K - r$ backbone evaluations after correction. LPIPS, PSNR, and boundary error use matched full rollback as the reference. Camera-trajectory errors are measured over the active chunk.

Receipt step r	Post-request backbone NFE ↓	Latent-ready latency (ms) ↓	Pixel-ready latency (ms) ↓	LPIPS ↓	PSNR ↑	Boundary error ↓	R_{dist} ↓	T_{dist} ↓
1	3	3535.8	7578.8	0.0643	23.13	0.0204	0.108	0.024
2	2	2748.8	7442.2	0.0831	20.51	0.0196	0.048	0.025
3	1	1980.1	6026.6	0.2694	18.39	0.0246	0.180	0.064

Later receipt steps leave fewer backbone evaluations to execute, but the $r = 3$ results show lower rollback fidelity than earlier corrections.

B.2 TEMPORAL-BOUNDARY SENSITIVITY

We fix receipt step $r = 2$ and vary the CST_T boundary $m \in \{1, 2, 3\}$ on HY-WM1.5. The 30 held-out prompts and matched trajectories are shared across boundaries. Table 8 reports prefix preservation, suffix fidelity, boundary continuity, and camera-trajectory error.

Table 8: Temporal-boundary sweep for CST_T on HY-WM1.5 with receipt step $r = 2$. Prefix LPIPS is measured on frames before m . Suffix metrics and camera-trajectory errors are measured on frames $\tau \geq m$. Boundary error is measured between frames $m - 1$ and m .

Boundary m	Prefix LPIPS ↓	Suffix LPIPS ↓	Suffix PSNR ↑	Boundary error ↓	R_{dist} ↓	T_{dist} ↓
1	0.0403	0.0701	24.74	0.0189	0.063	0.047
2	0.0443	0.0597	25.53	0.0208	0.049	0.093
3	0.0395	0.0578	24.69	0.0230	0.049	0.023

No boundary is best on every metric, and performance remains comparable across $m \in \{1, 2, 3\}$.

B.3 DIRECTED ACTION-TRANSITION BREAKDOWN

Table 9: Per-transition results for CST_R on HY-WM1.5 at receipt step $r = 2$. Rotation and translation errors are measured over the active chunk. LPIPS uses matched full rollback as the reference. The final row aggregates the six balanced transition groups.

Action transition	$R_{\text{dist}} \downarrow$	$T_{\text{dist}} \downarrow$	LPIPS to rollback $\downarrow$
Forward $\rightarrow$ Backward	0.002	0.028	0.0479
Forward $\rightarrow$ Yaw-left	0.078	0.031	0.1285
Backward $\rightarrow$ Forward	0.003	0.043	0.0593
Backward $\rightarrow$ Yaw-left	0.054	0.003	0.1042
Yaw-left $\rightarrow$ Forward	0.074	0.028	0.1434
Yaw-left $\rightarrow$ Backward	0.080	0.019	0.2654
All transitions	0.048	0.025	0.0831

B.4 HUMAN ANNOTATION INTERFACE

Figure 6 shows the blinded interface used for action-following and stale-frame annotation.

The interface is titled "CST_R CST_T" and shows "0 / 10 groups complete". It includes an "Export annotation CSV" button. Below the title bar, there are navigation buttons: "Previous", "Matched group", "Group 1 - four-segment schedule", and "Next".

The main section is titled "CST_R - THREE INTERRUPTIONS" and "Group 1 of 10". It displays a "Requested motion sequence: Forward $\rightarrow$ Yaw Left $\rightarrow$ Forward $\rightarrow$ Yaw Left" with "Interruptions at 5 s, 10 s, and 15 s".

Three video samples are shown, each with a "Pending" status and a "Frame 0 / 316 - 0.000 s" indicator. Each sample has a video player and a set of four action segments: "Segment 1 - 0-5 s", "Segment 2 - 5-10 s", "Segment 3 - 10-15 s", and "Segment 4 - 15-18 s". Each segment has a "Select" button and a "Next" button. The actions are "Forward", "Yaw Left", "Forward", and "Yaw Left" respectively.

Below the video samples is a "GROUP-LEVEL JUDGMENT" section. It includes a "Human preference" section with radio buttons for "sample 037", "sample 012", "sample 054", "sample 021", "sample 048", "sample 006", and "No clear preference". There is also a "Group notes (optional)" section with a text area and a "Save review" button.

Figure 6: Blinded interface for action-following and stale-frame annotation. Annotators inspect anonymized videos for a matched prompt and action schedule, score each action segment, and record a group-level preference. Method identities remain hidden.